

AI ASSISTED CODING

Assignment 6.1

Sameera anjum

2303A52466

Batch 40

1. Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Code:

```
N = int(input("Enter the value of N: "))
for num in range(2, N + 1, 2):
    print(num)
```

Output:

```
Enter the value of N: 10
2
4
6
8
10
```

Analysis: The code prompts the user for a value N, then uses a for loop with `range(2, N + 1, 2)` to iterate through all even numbers from 2 up to N (inclusive). Each even number is printed. This approach is efficient because it skips odd numbers entirely by using a step of 2.

2. Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Code:

```
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
even_count = 0
odd_count = 0
for num in numbers:
    if num % 2 == 0:
        even_count += 1
    else:
        odd_count += 1
print(f"Even numbers: {even_count}")
print(f"Odd numbers: {odd_count}")
```

Output:

```
Enter numbers separated by space: 1 3 4 2 5 4 3 6 7
Even numbers: 4
Odd numbers: 5
```

Analysis: The code takes a space-separated list of numbers as input, converts them to integers, and iterates through each number. It uses the modulo operator % to check if a number is even ([num % 2 == 0](#)) or odd, incrementing the respective counters. The final counts are printed. This method is straightforward and works for any list of integers.

3. Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Code:

```

class User:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

    def is_valid_age(self):
        # Age should be a positive integer and reasonable (e.g., 0 < age < 120)
        return isinstance(self.age, int) and 0 < self.age < 120

    def is_valid_email(self):
        # Simple email validation: contains '@' and '.' after '@'
        if '@' in self.email:
            local, _, domain = self.email.partition('@')
            return '.' in domain and local != '' and domain != ''
        return False

    def validate(self):
        if not self.is_valid_age():
            print("Invalid age.")
        elif not self.is_valid_email():
            print("Invalid email.")
        else:
            print("User is valid.")

user = User("Alice", 25, "alice@example.com")
user.validate()

```

Output:

```
User is valid.
```

Analysis: The `User` class has attributes for name, age, and email. The `is_valid_age` method checks if age is an integer between 1 and 119. The `is_valid_email` method checks for the presence of '@' and a '.' in the domain part, ensuring basic email format validity. The `validate` method prints whether the user is valid based on these checks. This provides simple but effective validation using conditional statements.

4. Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Code:

```

class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks # marks should be a list of numbers

    def total_marks(self):
        return sum(self.marks)

    def average_marks(self):
        if len(self.marks) == 0:
            return 0
        return sum(self.marks) / len(self.marks)
student = Student("Bob", "101", [85, 90, 78])
print("Total Marks:", student.total_marks())
print("Average Marks:", student.average_marks())

```

Output:

```

Total Marks: 253
Average Marks: 84.33333333333333

```

Analysis: The [Student](#) class stores name, roll number, and a list of marks. The [total_marks](#) method returns the sum of all marks, while [average_marks](#) returns the average, handling the case of an empty marks list by returning 0. This design allows easy calculation of total and average marks for any student.

5. Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Code:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrawn: {amount}")
        else:
            print("Invalid or insufficient funds for withdrawal.")

    def display_balance(self):
        print(f"Current Balance: {self.balance}")
```

```
while True:
    print("\n1. Deposit\n2. Withdraw\n3. Display Balance\n4. Exit")
    choice = input("Enter your choice: ")
    if choice == '1':
        amt = float(input("Enter amount to deposit: "))
        account.deposit(amt)
    elif choice == '2':
        amt = float(input("Enter amount to withdraw: "))
        account.withdraw(amt)
    elif choice == '3':
        account.display_balance()
    elif choice == '4':
        print("Exiting...")
        break
    else:
        print("Invalid choice. Try again.")
```

Output:

```
1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 1
Enter amount to deposit: 5000
Deposited: 5000.0

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 3
Current Balance: 5000.0

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 2
Enter amount to withdraw: 5000
Withdrawn: 5000.0
```

0

Analysis:

- **Class Design:** The [BankAccount](#) class models a bank account with attributes for the owner's name and balance. It provides methods for depositing, withdrawing, and displaying the balance.
- **Deposit Method:** Validates that the deposit amount is positive before adding it to the balance. If the amount is invalid, it prints an error message.

- **Withdraw Method:** Checks that the withdrawal amount is positive and does not exceed the current balance. If the conditions are met, it deducts the amount; otherwise, it prints an error message for invalid or insufficient funds.
- **Display Balance:** Simply prints the current balance.
- **Interactive Loop:** Uses a while True loop to repeatedly prompt the user for actions (deposit, withdraw, display balance, exit). User input determines which method is called. The loop continues until the user chooses to exit.
- **Conditional Statements:** Used throughout to validate user choices and transaction amounts, ensuring the system behaves correctly and securely.